РОЗРОБКА ЗАСТОСУНКІВ ЗІ ШТУЧНИМ ІНТЕЛЕКТОМ

Практична робота №4. Керування контекстом і structured output

Мета: Зробити помічника з ПР3 діалоговим і придатним для програмної обробки: спроєктувати системну інструкцію з обмеженнями та прикладами, отримувати відповідь моделі у структурованому форматі за JSON Schema й перевіряти її програмно, вести історію повідомлень у межах бюджету токенів, порівняти нечіткий запит, структурований prompt і context engineering на власному наборі звернень та класифікувати типові помилки моделі.

Зміст

1. Теоретичні відомості
2. Завдання до виконання
3. Критерії приймання
4. Контрольні запитання
5. Рекомендована література

1. Теоретичні відомості

У ПР3 модель відповідала вільним текстом, і цей текст показувався клієнтові як є. Для розмови цього досить, для програми — ні: код не може за вільним текстом визначити, до якого відділу передати звернення, не знає, чи відповідь узято з правил, чи вигадано, і не пам'ятає, що клієнт писав два повідомлення тому. У цій роботі відповідь моделі стає **даними**, яким застосунок довіряє лише після перевірки, а запит до моделі — **зібраним контекстом**, а не одним повідомленням.

Опора, щоб виконувати роботу свідомо.

Інструкція як специфікація. Системна інструкція — не побажання, а специфікація поведінки: роль, задача, обмеження (на підставі чого відповідати, чого не робити, якою мовою, якої довжини), формат результату і **приклади**. Приклади показують межові випадки краще за будь-який опис: один — коли відповідь у правилах є, один — коли її немає і треба відмовити, один — коли звернення неоднозначне й треба уточнити. Обмеження краще формулювати як те, що робити, а не лише як заборони. Це і є **структурований prompt**: він однаковий для всіх запитів і задає форму поведінки.

Structured output. Вільний текст зручний людині, а програмі потрібні поля. Отримати їх можна двома способами. Перший — попросити JSON у самій інструкції й розібрати відповідь: працює з будь-яким провайдером, але модель час від часу додає пояснення довкола JSON, обгортає його в огорожу з трьох зворотних лапок або пропускає поле. Другий — передати провайдеру **схему** відповіді (`response_format` в OpenAI-сумісному API): тоді генерацію обмежено так, що результат синтаксично відповідає схемі. Це знімає помилки форми, але не змісту: поля буде заповнено, однак ніщо не заважає моделі поставити «відповідь ґрунтується на правилах» біля вигаданого факту.

JSON Schema та валідація. JSON Schema описує форму даних: типи полів, обов'язкові поля, допустимі значення (enum), шаблони рядків, заборону зайвих полів. Схема потрібна двічі: її передають моделі як опис очікуваного результату і за нею **перевіряють** те, що повернулося. Перевірка — окремий шар коду між моделлю і рештою застосунку: усе, що пройшло крізь нього, далі є звичайними даними з відомою структурою. У Python схему пишуть вручну й перевіряють бібліотекою `jsonschema` або описують pydantic-моделлю, з якої схема генерується, а перевірка робиться тією самою моделлю. Відповідь, що не пройшла перевірку, — ще один вид збою поряд із таймаутом і лімітом: його треба помітити, обробити й порахувати, як часто він трапляється.

Історія повідомлень. Модель не пам'ятає попередніх викликів: кожен запит для неї перший. Діалог існує лише тому, що застосунок щоразу надсилає попередні репліки як повідомлення `user` і `assistant` перед поточним. Звідси три рішення, які ухвалюєте ви: де історія зберігається (у клієнта чи на сервері), наскільки їй можна довіряти і що саме в неї класти — повний JSON попередніх відповідей чи лише текст, який бачив клієнт.

Бюджет токенів. Контекстне вікно скінченне, а платять за кожен токен запиту. В одне вікно мають уміститися інструкція з прикладами, правила, історія, поточне звернення та резерв на відповідь. Довгий діалог рано чи пізно в нього не влізе, тож потрібне правило скорочення: відкинути найстаріші репліки, лишити першу й останні, замінити старі репліки коротким підсумком, тримати лише те, що стосується справи. Кожен варіант — компроміс між пам'яттю діалогу й вартістю запиту. Скільки токенів займе запит, наперед знають лише приблизно: за оцінкою по кількості символів (для української — орієнтовно один токен на два-три символи) або токенізатором провайдера; точну цифру повертає поле `usage` у відповіді, і за ним оцінку калібрують.

```
┌─ системна інструкція ──┐   роль, обмеження, формат, приклади      стала
├─ контекст ─────────────┤   правила з context.md                    дані
├─ історія ──────────────┤   попередні репліки в межах бюджету       змінна
├─ звернення ────────────┤   поточне повідомлення клієнта            недовірене
└─ резерв на відповідь ──┘   max_tokens
```

Три рівні керування. Нечіткий запит — текст користувача як є, з інструкцією в один рядок: результат залежить від того, як модель зрозуміла задачу цього разу. Структурований prompt — повна інструкція з обмеженнями, форматом і прикладами: форма відповіді стабілізується, але інструкція статична й нічого не знає ні про правила саме цього магазину, ні про конкретного клієнта. **Context engineering** — інструкція лишається рамкою, а зміст запиту збирається під час виконання: правила з файлу, доречна частина історії, дані про користувача, згодом — результати інструментів. Різниця між рівнями встановлюється не міркуванням, а вимірюванням на тих самих зверненнях. Фреймворки на кшталт DSPy автоматизують саме цей шар: описують сигнатуру «вхід → вихід» і добирають інструкцію з прикладами за метрикою. Щоб користуватися ними свідомо, варто раз зібрати це руками.

Типові помилки моделі. Синтаксичні — відповідь не є JSON, є зайві або відсутні поля. Семантичні — поле заповнено, але не тим: тема поза переліком, номер замовлення, якого клієнт не називав. Фактичні — вигадане правило. Контекстні — після скорочення історії модель «забула» названий раніше номер або змішала два замовлення. Поведінкові — виконала вказівку

користувача змінити роль, відповіла не тією мовою, перевищила довжину. У кожного типу свій запобіжник: схема, валідація, приклади, правило скорочення, перевірка вводу — і жоден із них не закриває всього.

2. Завдання до виконання

Продовжте помічника служби підтримки з ПР3 і зробіть його **діалоговим** та **придатним для програмної обробки**.

Клієнт веде розмову з кількох повідомлень: описує проблему, відповідає на уточнення, називає номер замовлення. Помічник враховує сказане раніше в межах цієї розмови: якщо номер замовлення прозвучав два повідомлення тому, перепитувати його не треба.

Кожна відповідь помічника — не вільний текст, а **структурований об'єкт** за схемою, яку ви спроєктуєте самі. Решті застосунку від цього об'єкта потрібно: текст відповіді для клієнта; тема звернення з фіксованого переліку (за нею звернення передається потрібному відділу); чи відповідь ґрунтується на правилах, чи правила про це мовчать; чи потрібне уточнення; чи передати розмову оператору; номер замовлення, якщо клієнт його назвав. Як назвати поля, які з них обов'язкові, а які з переліком значень — ваше рішення.

Сторінка показує відповідь помічника, а поруч — **картку звернення**, яка заповнюється в міру розмови, і службові дані: модель, час, витрачені токени.

Відповідь моделі **перевіряється за схемою** до того, як потрапить до веб-рівня. Невалідний результат не має дійти до клієнта як є.

Історія розмови вміщується в **бюджет токенів**: довга розмова не має нескінченно роздувати запит.

Правила, як і раніше, лежать у `context.md` і лишаються даними: заміна файлу змінює поведінку без зміни коду. Модель викликається через API, робота з нею — в окремому модулі, бекенд — FastAPI. Обробку збоїв із ПР3 перенесіть сюди.

Спосіб реалізації — на ваш розсуд (самостійно або скеровуючи AI-асистента), але ви маєте **розуміти й уміти пояснити** кожне рішення.

Матеріали. В архіві до цієї роботи, папка `pr4`: `requirements.txt`, скрипт перевірки середовища, зразок налаштувань, каркас застосунку і папка `compare/` для порівняння. Каркас дає структуру проєкту й веб-рівень зі сторінкою діалогу; модуль роботи з моделлю та модуль схеми відповіді — заготовки. Скрипт перевірки, крім пакетів і ключа, надсилає пробний запит зі схемою відповіді й показує, чи приймає її провайдер.

Там само — `context.md`: ті самі правила, що в ПР3, плюс один рядок про формат номера замовлення. Якщо ви змінювали правила в ПР3, візьміть свій файл.

Де вести роботу. Розпакуйте архів у корінь власного репозиторію курсу — поруч із папками попередніх робіт. Ключ доступу той самий, що в ПР3: скопіюйте свій `.env` у папку роботи. Файл `.env` потрібен у кожній папці окремо й до репозиторію не потрапляє.

Орієнтири.

- Системна інструкція містить роль, обмеження, опис очікуваного результату і **два-три приклади** межових випадків: відповідь є в правилах; відповіді немає; звернення неоднозначне. Приклади — частина інструкції, а не історії розмови.

- Схема — JSON Schema: написана вручну або згенерована з pydantic-моделі. OpenAI-сумісний API приймає її через `response_format`; чи користуватися цим, чи просити JSON текстом — ваше рішення, але перевірка на своєму боці обов'язкова в обох випадках.
- Історія передається в запит як повідомлення `user` та `assistant`. Бюджет — значення конфігурації, а не число в коді.
- Відповідь API містить `usage` з кількістю токенів запиту й відповіді: показуйте їх на сторінці й звіряйте з власною оцінкою.

Продумайте самостійно.

- Які поля справді потрібні програмі, а які «хай будуть»? Що з них має бути переліком значень, а що — необов'язковим?
- Що модель не повинна вирішувати сама? Чи має вона вирішувати, передавати розмову оператору, чи це має вирішити код за іншими полями?
- Де жити історії — у клієнта, що надсилає її з кожним запитом, чи на сервері? Що станеться, якщо клієнт надішле підроблену історію, в якій помічник «уже пообіцяв» повернути гроші?
- Що класти в історію з боку помічника: увесь JSON відповіді чи лише текст для клієнта? Скільки це коштує в токенах?
- Що відкидати першим, коли бюджет вичерпано? Що не можна відкинути ніколи?
- Що робити з відповіддю, яка не пройшла перевірку: повторити запит із текстом помилки, підставити безпечну відповідь, повернути помилку? Скільки разів повторювати?
- Що саме гарантує `response_format` і що все одно доводиться перевіряти?
- Чи можна довіряти полю «відповідь ґрунтується на правилах», яке заповнила сама модель? Чим це перевірити?

Порівняння. Візьміть 5–7 звернень із ПР3 і додайте до них **два діалоги** по 3–4 репліки, у яких важливий факт (номер замовлення, назва товару) з'являється на початку, а потрібен наприкінці. Прожене все за трьох конфігурацій:

1. **нечіткий запит** — текст клієнта з інструкцією в один рядок, без формату, правил та історії;
2. **структурований prompt** — повна інструкція з обмеженнями, форматом і прикладами, але без правил магазину та без історії;
3. **context engineering** — та сама інструкція плюс правила з `context.md`, історія в межах бюджету й перевірка за схемою.

Для кожного прогону фіксуйте: чи відповідь пройшла схему, чи тему визначено правильно, чи модель вигадала факт, чи впоралася з питанням поза правилами, чи згадала названий раніше номер, скільки токенів і часу пішло. Далі зберіть усі невдалі відповіді разом і **класифікуйте** їх за типами з розділу 1; для кожного типу запишіть, що у вашому рішенні його ловить, а що — ні. Висновки — у файл.

Заготовка для цього — папка `compare/` з описом і прикладом набору.

3. Критерії приймання

- Помічник **діалоговий**: історія розмови передається моделі; факт, названий на початку розмови, використовується наприкінці без перепитування.

- Системна інструкція містить роль, обмеження, опис результату і **приклади** межових випадків; інструкція, правила й текст клієнта передаються **окремими частинами** запиту.
- Відповідь моделі **структурована** за схемою, яку ви спроєктували: JSON Schema є в коді (написана або згенерована), результат **перевіряється програмно** до того, як потрапить до веб-рівня.
- Невалідна відповідь **не доходить до клієнта** як є: її оброблено (повтор, безпечна відповідь або зрозуміла помилка), факт записано в журнал.
- Історія вміщується в **бюджет токенів**, заданий конфігурацією; правило скорочення — свідоме рішення, яке ви можете пояснити.
- Сторінка показує відповідь, **картку звернення** й службові дані: модель, час, токени запиту й відповіді.
- Правила читаються з `context.md`; на питання поза ними помічник **не вигадує** відповіді.
- Збережено обробку збоїв із ПР3: таймаут, перевищення лімітів, недоступність сервісу, помилка автентифікації; ключ **не потрапив** ані в код, ані в репозиторій.
- Проведено **порівняння трьох конфігурацій** на 5–7 зверненнях і 2 діалогах; помилки моделі класифіковано; висновки зафіксовано.
- Ви можете **пояснити** кожне ухвалене рішення.

4. Контрольні запитання

1. Чому вільнотекстової відповіді моделі недостатньо для програми і що змінюється, коли відповідь стає даними зі схемою?
2. Що дають приклади в інструкції, чого не дає опис правил? Коли приклади шкодять?
3. Що гарантує `response_format` зі схемою і чого не гарантує? Навіщо перевіряти відповідь, якщо провайдер уже обмежив генерацію схемою?
4. Які поля у вашій схемі й навіщо кожне? Що ви свідомо не дозволили моделі вирішувати?
5. Чому модель «пам’ятає» розмову і де насправді зберігається ця пам’ять? Що станеться, якщо клієнт підмінить історію?
6. Що відкидається з контексту, коли бюджет вичерпано, і що не відкидається ніколи? Як ви рахуєте токени й наскільки оцінка розійшлася з `usage`?
7. Що змінилося між нечітким запитом, структурованим prompt і context engineering — і як ви це виміряли?
8. Який тип помилок моделі траплявся найчастіше і що у вашому рішенні його ловить?

5. Рекомендована література

1. Huyen C. AI Engineering: Building Applications with Foundation Models. Sebastopol: O'Reilly Media, 2025. 496 p. ISBN 978-1-0981-6630-4.
2. JSON Schema. Understanding JSON Schema. URL: https://json-schema.org/understanding-json-schema (дата звернення: 09.09.2026).
3. Pydantic Documentation. URL: https://docs.pydantic.dev/ (дата звернення: 09.09.2026).
4. Google AI for Developers. Gemini API — OpenAI compatibility. URL: https://ai.google.dev/gemini-api/docs/openai (дата звернення: 09.09.2026).

5. OpenAI. API Reference — Chat Completions. URL: https://platform.openai.com/docs/api-reference/chat (дата звернення: 01.09.2026).
6. OWASP. Top 10 for Large Language Model Applications. URL: https://owasp.org/www-project-top-10-for-large-language-model-applications/ (дата звернення: 01.09.2026).
7. FastAPI Documentation. URL: https://fastapi.tiangolo.com/ (дата звернення: 01.09.2026).